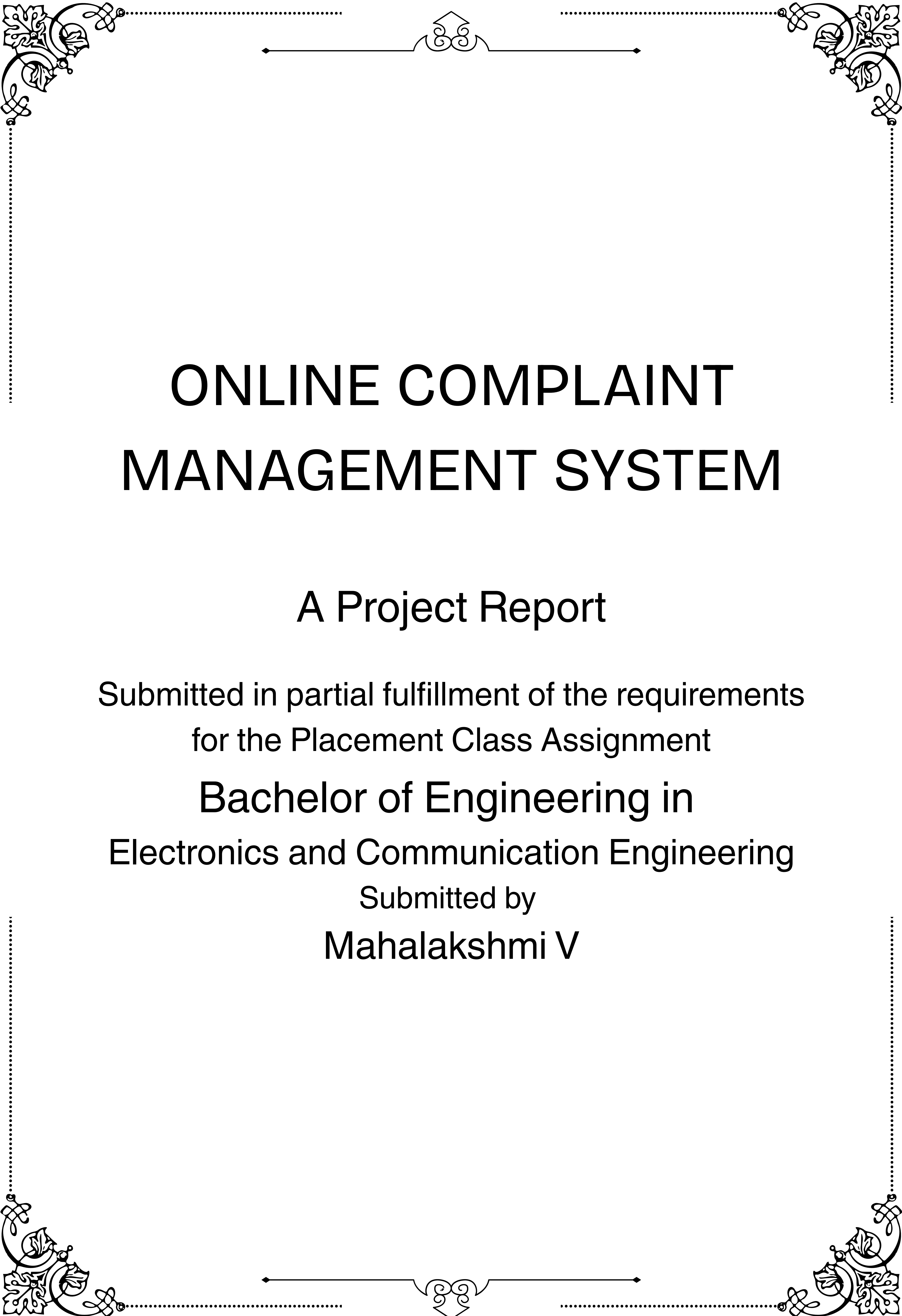




ONLINE COMPLAINT MANAGEMENT SYSTEM

A Project Report

Submitted in partial fulfillment of the requirements
for the Placement Class Assignment

Bachelor of Engineering in
Electronics and Communication Engineering

Submitted by
Mahalakshmi V

1. INTRODUCTION

The Complaint Management System (CMS) is a robust, enterprise-oriented web application engineered to digitize and optimize the end-to-end complaint handling lifecycle within an organization. The system facilitates structured interaction between multiple stakeholders—namely end users, administrators, and IT personnel—by enforcing clearly defined roles, responsibilities, and access boundaries.

From a technical perspective, the application is designed using a layered architecture that separates presentation logic, business logic, and data persistence concerns. This architectural approach improves maintainability, scalability, and testability while aligning with industry-standard software engineering practices.

The CMS leverages Spring Boot as its core backend framework, exposing RESTful services that interact with a relational database powered by MySQL, which is actively running and configured as the primary persistence layer. All transactional data—including user credentials, complaint records, assignments, and status transitions—are securely stored and managed within the MySQL database.

By eliminating manual intervention and fragmented communication channels, the system ensures real-time visibility into complaint status, enforces accountability through role-based workflows, and provides a scalable foundation for future enterprise enhancements.

2. Problem Statement

In conventional organizational environments, complaint handling is often performed using unstructured mechanisms such as emails, phone calls, spreadsheets, or paper-based logs. These approaches introduce several systemic inefficiencies and technical limitations:

- Absence of a centralized and normalized data repository
- Inability to track complaint status in real time
- Lack of automated ownership assignment and escalation
- High risk of data redundancy, inconsistency, and loss
- Minimal auditability and reporting capability
- No enforced access control or role segregation

From a systems engineering standpoint, these limitations result in poor data integrity, delayed resolution cycles, and reduced service quality. Furthermore, the absence of a structured backend and database-driven workflow makes it difficult to scale operations or integrate analytical insights.

The problem addressed by this project is the need for a secure, role-aware, database-backed complaint management platform that can automate complaint intake, enforce workflow rules, and maintain persistent, queryable records using a relational database system.

3. Objectives of the System

The primary objectives of the Complaint Management System are:

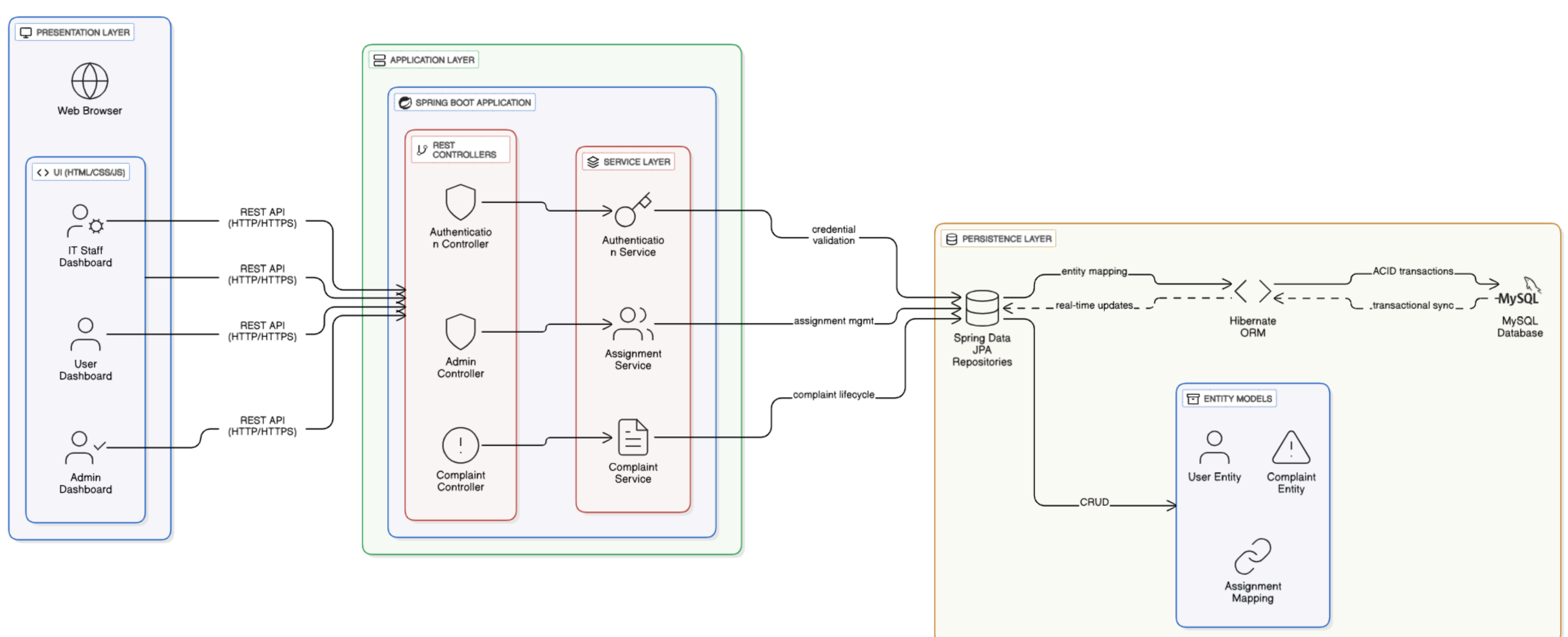
- To provide a centralized digital platform for complaint submission and management
- To enforce role-based access for Users, Administrators, and IT Staff
- To enable real-time complaint tracking and status updates
- To automate complaint assignment to IT personnel
- To maintain persistent and structured data storage using MySQL
- To ensure scalability and ease of future enhancements

4. System Architecture Overview

The system follows a three-tier architecture:

- **Presentation Layer** – Frontend user interfaces
- **Application Layer** – Backend business logic and REST APIs
- **Data Layer** – MySQL relational database

This separation of concerns improves maintainability and allows independent scaling of system components.



5. Technology Stack

The Complaint Management System is implemented using a modern full-stack technology stack that emphasizes modularity, performance, and enterprise readiness.

5.1 Backend Technologies

- Java: Serves as the primary programming language, offering platform independence, strong type safety, and mature ecosystem support.
- Spring Boot: Acts as the core application framework, enabling rapid development through auto-configuration, embedded server support, and dependency injection.
- Spring MVC: Manages HTTP request routing, controller mapping, and REST endpoint exposure.
- Spring Data JPA: Abstracts database interaction layers and minimizes boilerplate code for CRUD operations.
- Hibernate ORM: Implements object-relational mapping (ORM), translating Java entity models into relational MySQL tables.
- RESTful Web Services: Provides stateless, standardized communication between frontend clients and backend services using HTTP verbs.

5.2 Frontend Technologies

- HTML5: Defines the structural layout of user interfaces.
- CSS3: Handles styling, layout responsiveness, and UI consistency.
- JavaScript: Implements client-side logic, form validation, and asynchronous API consumption.

5.3 Database Layer

- MySQL (Running Database Engine)
 - Serves as the centralized relational database management system.
 - Ensures ACID-compliant transactions, referential integrity, and concurrency control.
 - Integrated using Hibernate with MySQL8Dialect for optimized SQL generation.
 - Schema evolution is managed automatically using JPA configuration (ddl-auto=update).

6. Database Design and Data Flow

6.1 Database Configuration

The application is configured to connect to a MySQL database through application.properties. Hibernate manages schema creation and updates automatically using ddl-auto=update.

6.2 Key Database Entities

- User Table: Stores user, admin, and IT staff credentials along with role information
- Complaint Table: Stores complaint details, timestamps, and status
- Assignment Mapping: Links complaints to IT staff members

6.3 Data Flow

1. User submits complaint
2. Complaint data is persisted in MySQL
3. Admin assigns complaint
4. Assignment updates database records
5. IT staff updates resolution status
6. Changes are reflected across all roles

7. Functional Module Description and Working Process

The Complaint Management System operates through tightly integrated functional modules, each governed by role-based access control and coordinated through RESTful backend services.

7.1 User Module – Technical Workflow

- User authentication requests are submitted via HTTP POST and validated against encrypted credential records stored in MySQL.
- Upon successful authentication, a session-aware response grants access to user-specific endpoints.
- Complaint submission triggers backend validation, entity mapping, and persistence into the complaint table with an initial lifecycle state (e.g., Pending).
- Complaint retrieval operations use filtered database queries to ensure users can access only their own records.

7.2 Admin Module – Complaint Orchestration

- Admin authentication includes role verification to enable elevated privileges.
- The admin dashboard aggregates complaint data through optimized database joins and repository queries.
- Complaint assignment updates foreign-key relationships between complaint and IT staff entities.
- Status transitions (Assigned, In Progress, Resolved) are enforced through controlled backend logic to maintain data consistency.

7.3 IT Staff Module – Resolution Lifecycle

- IT staff access is restricted to assigned complaint records using role- and ID-based query filtering.
- Complaint resolution updates propagate to the database in real time.
- Status changes are immediately visible across user and admin views, ensuring synchronized system state.

This structured workflow ensures a deterministic complaint lifecycle backed by transactional database operations.

8. Security and Role-Based Access Control

The system implements role-based authorization to ensure that:

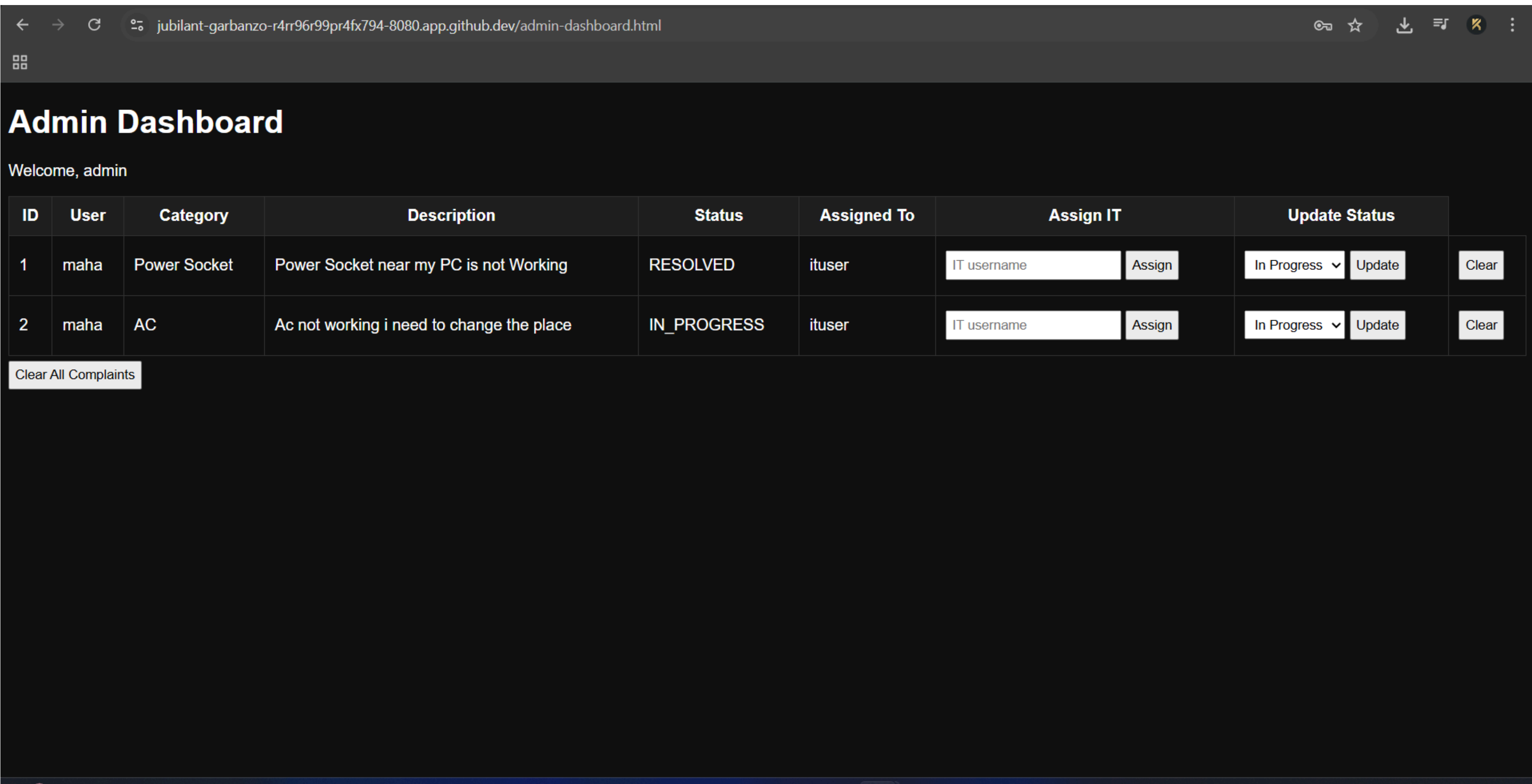
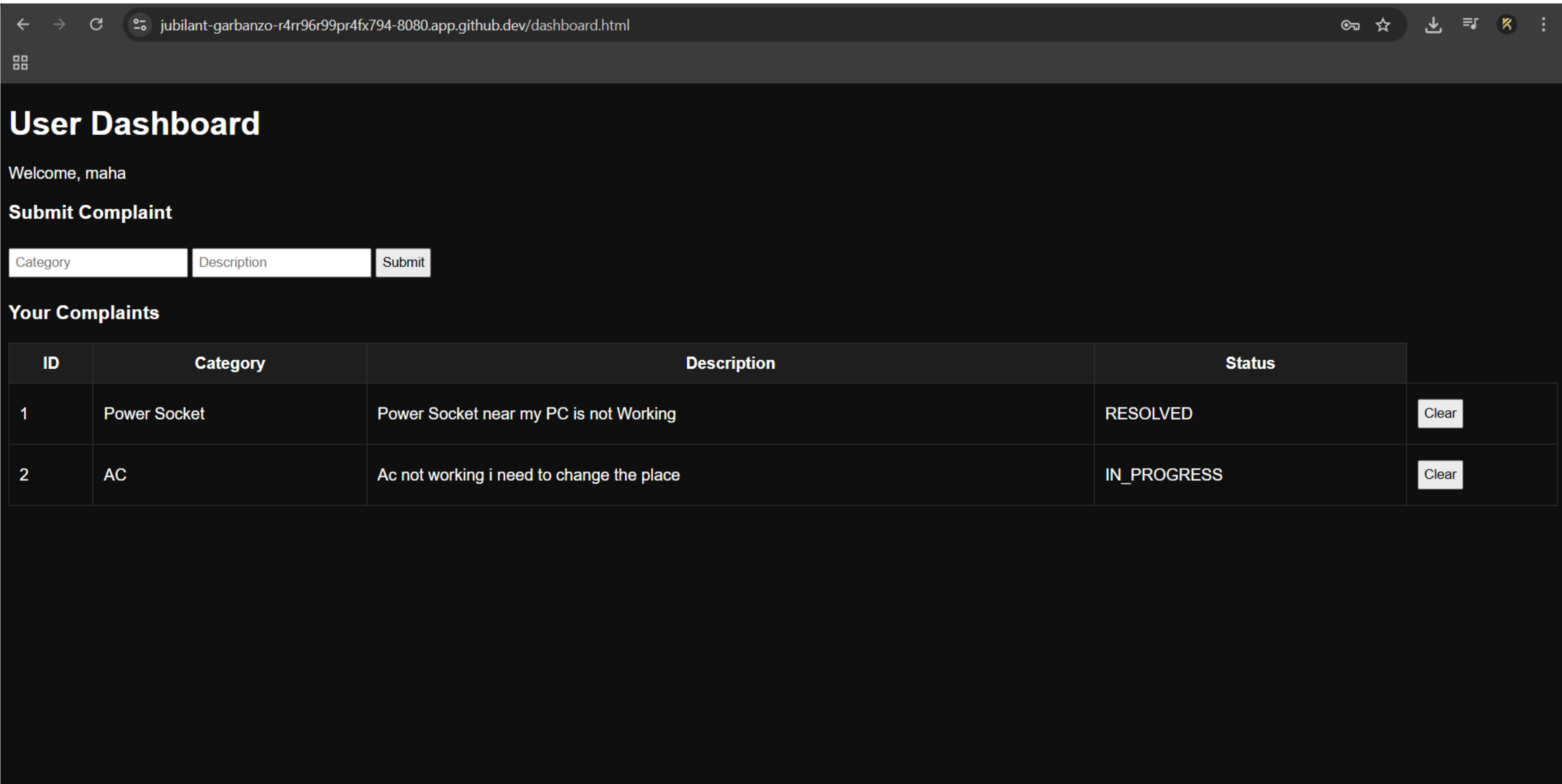
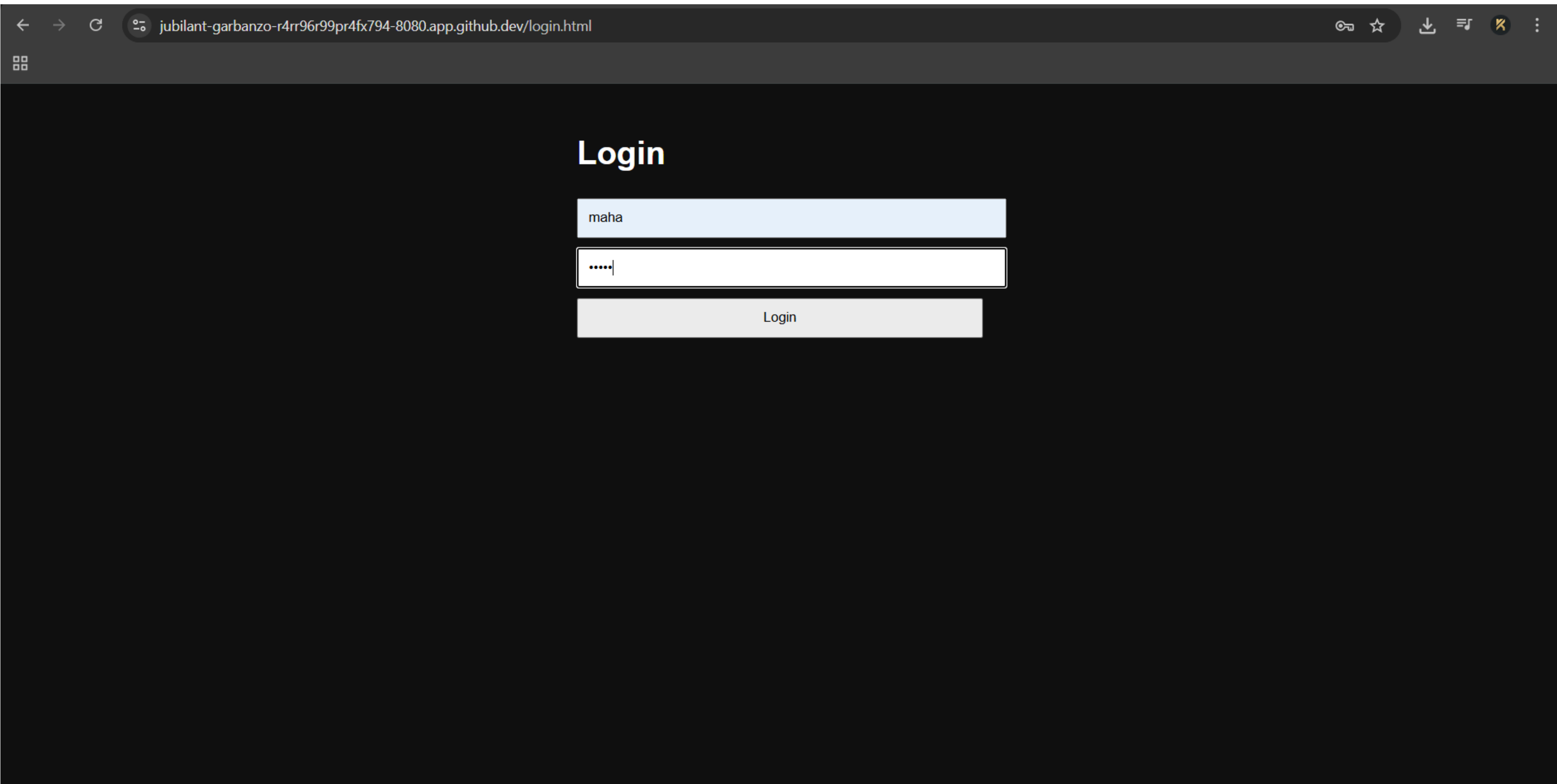
- Users access only their complaints
- Admins have full system visibility
- IT staff access only assigned tasks

Authentication is enforced at the controller level, preventing unauthorized access to protected resources.

9. Error Handling and Validation

- Backend validation ensures required fields are not null
- Proper HTTP response codes are returned for success and failure cases
- Exception handling prevents application crashes
- Input sanitization reduces invalid data persistence

10. Output Interface



11. Testing and Execution

- Application tested using local MySQL instance
- API endpoints validated through browser and REST clients
- Database transactions verified through real-time table updates

12. Future Enhancements

- Email and SMS notification system
- Complaint priority classification
- Analytics and reporting dashboard
- Integration with mobile applications
- JWT-based authentication

13. Conclusion

The Complaint Management System demonstrates a robust and scalable implementation of a real-world enterprise application. By leveraging Spring Boot, RESTful services, and a running MySQL database, the system delivers a secure, efficient, and transparent complaint-handling workflow. The modular architecture and clean separation of concerns make it suitable for both academic evaluation and real-world deployment.